Capstone Project : Cloud-Native Omni-Channel Retail Analytics Platform

Background

Modern retail companies operate across multiple channels—physical stores, e-commerce websites, mobile apps, and third-party marketplaces. Each channel generates massive volumes of transactional, customer, inventory, and behavioral data.

However, this data is often siloed, inconsistent, and delayed, making it difficult for business teams to answer critical questions like:

- Which products are performing best across channels?
- How do promotions impact sales and margins?
- Are inventory levels aligned with demand?
- What customer segments drive repeat purchases?

A modern, cloud-native analytics platform is required to unify omni-channel data and deliver reliable, real-time, and historical insights.

Business Context

Key stakeholders include:

- **Merchandising Teams**
  - product performance and pricing insights
- **Marketing Teams**
  - campaign effectiveness and customer segmentation
- **Supply Chain Teams**
  - inventory optimization and stock-out prevention
- **Store Operations**
  - store-level performance and footfall analysis
- **Leadership**
  - revenue, growth, and profitability dashboards

They need an analytics platform that can:

- Integrate data from POS systems, e-commerce platforms, CRM, and inventory systems
- Handle structured and semi-structured data
- Support batch and near real-time ingestion
- Enable self-service analytics and BI
- Be scalable, reliable, and cloud-native

Objective

Build an end-to-end modern data analytics platform for an omni-channel retail company to:

© NIIT 2025-26

All Information within this document is Intellectual property of NIIT (NIIT Ltd). No part of this document or the program design or program structure mentioned within can be shared or used within any organization without the permission of NIIT (NIIT Ltd).

- Ingest raw retail data from multiple sources using incremental ingestion patterns
- Clean, transform, and enrich data using PySpark with deterministic deduplication
- Store curated data in Delta Lake (Bronze/Silver/Gold layers) with proper partitioning
- Orchestrate pipelines using Apache Airflow DAGs and Azure Data Factory with parameterization and metadata logging
- Enable analytical queries via SQL endpoints through governed Gold-layer views only
- Implement CI/CD pipelines with Git integration and environment separation (DEV / TEST / PROD)
- Enforce monitoring, alerting, and SLA tracking across all pipeline tasks

Dataset Description

The project uses multiple retail datasets simulating real enterprise systems.

1. Sales Transactions

- transaction_id
- order_date, order_time
- channel (Store / Online / Mobile)
- store_id
- product_id
- customer_id
- quantity
- unit_price
- discount
- total_amount
- payment_type

2. Product Master

- product_id
- product_name
- category
- sub_category
- brand
- cost_price
- selling_price

3. Store Master

- store_id

All Information within this document is Intellectual property of NIIT (NIIT Ltd). No part of this document or the program design or program structure mentioned within can be shared or used within any organization without the permission of NIIT (NIIT Ltd).

- store_name
- city
- state
- store_type (Mall / Standalone / Warehouse)

4. Customer Data

- customer_id
- gender
- age
- loyalty_status
- signup_channel
- city

5. Inventory Data

- product_id
- store_id
- stock_on_hand
- reorder_level
- last_updated

Solution Architecture

```
Source Systems

(POS | E-commerce | Inventory | CRM)

        |

        v

Azure Data Factory (Incremental Ingestion + Metadata Logging)

        |

        v

Delta Lake (Bronze – Raw, partitioned by order_date + channel)

        |

        v

Databricks (PySpark Transformations + Validation Notebooks)

        |
```


All Information within this document is Intellectual property of NIIT (NIIT Ltd). No part of this document or the program design or program structure mentioned within can be shared or used within any organization without the permission of NIIT (NIIT Ltd).

```
        v

Delta Lake (Silver – Cleaned, deduplicated, enriched)

        |

        v

Delta Lake (Gold – Dimensional Model + Pre-aggregated KPIs)



Orchestration Layer:

Apache Airflow DAG  -->  Task dependencies: Ingest > Bronze > Silver > Validate >
Gold

                          Retry config | SLA monitoring | Failure alerting



CI/CD Layer:

Git Repository  -->  CI Pipeline (lint / validate / test)  -->  CD Pipeline (DEV >
TEST > PROD)



Environment Separation:

DEV workspace  |  TEST workspace  |  PROD workspace

(separate storage containers, Delta schemas, Airflow variables per environment)
```

Solution Outline

Layer	Tools / Technologies	Key Tasks
Ingestion	Azure Data Factory	Incremental ingestion via watermark; parameterized pipelines; metadata logging
Profiling	Pandas, PySpark	Data quality checks, nulls, duplicates, outliers
Cleaning & Transformation	Databricks (PySpark)	Deterministic deduplication; standardize formats; derive net_sales, gross_margin

All Information within this document is Intellectual property of NIIT (NIIT Ltd). No part of this document or the program design or program structure mentioned within can be shared or used within any organization without the permission of NIIT (NIIT Ltd).

Storage	Delta Lake	Bronze / Silver / Gold with partitioning by order_date and channel
Orchestration	Apache Airflow + ADF	Airflow DAG with task dependencies, retry config, SLA monitoring; ADF for cloud-native ingestion
CI/CD	Git + CI/CD Pipeline	Version control for notebooks, DAGs, SQL scripts, ADF templates; automated promotion to DEV / TEST / PROD
Environment	DEV / TEST / PROD	Separate storage containers, Delta schemas, and Airflow variables per environment
Validation	PySpark Tests	Executable business rule checks, reject tables, rule logging
Monitoring	Airflow + Alerting	DAG failure alerts, email/Slack notifications, SLA breach detection, retry exhaustion tracking

Capstone Tasks for Learners

1. Raw Data Audit & Profiling

- Identify missing values, duplicates, invalid transactions
- Detect negative sales or zero prices
- Generate profiling reports

2. Schema Design & Storage Strategy

- Define data types and keys
- Implement partitioning by order_date and channel
- Store data in Delta format

3. Data Cleaning & Enrichment

- Handle null customer demographics
- Remove duplicate transactions using deterministic window function deduplication
- Standardize date/time formats

Derive new fields:

- net_sales

All Information within this document is Intellectual property of NIIT (NIIT Ltd). No part of this document or the program design or program structure mentioned within can be shared or used within any organization without the permission of NIIT (NIIT Ltd).

- gross_margin
- Order Value Band (Low / Medium / High)

4. PySpark Transformations

- Join sales with product, store, and customer data
- Create channel-wise and store-wise aggregates
- Build fact and dimension-style tables (fact_sales, dim_customer, dim_product, dim_store)

5. Pipeline Orchestration – Airflow DAG

Design and implement an Apache Airflow DAG to orchestrate the full pipeline:

- task_1: incremental_ingestion — trigger ADF pipeline or file sensor
- task_2: bronze_load — run Databricks Bronze notebook
- task_3: silver_transformation — run Databricks Silver notebook
- task_4: validation_checks — run Databricks Validation notebook
- task_5: gold_aggregation — run Databricks Gold notebook

- Configure retries=3, retry_delay=5 min, SLA=2 hours at DAG level
- Configure on_failure_callback for Slack/email alerting

6. CI/CD Pipeline Setup

Implement a CI/CD pipeline with Git integration:

- Create Git repository with folders for: notebooks/, dags/, sql/, adf_templates/
- Configure CI pipeline: notebook syntax check, SQL lint, DAG import validation, transformation unit tests
- Configure CD pipeline with DEV → TEST → PROD promotion stages

- Parameterize environment configs: storage paths, schema names, connection strings
- Implement manual approval gate for PROD deployment

7. Environment Separation

- Create three isolated workspaces: DEV, TEST, PROD
- Configure separate storage containers and Delta schema namespaces per environment
- Set environment-specific Airflow variables to prevent cross-environment data access
- Document promotion process: feature branch → DEV → TEST → PROD

8. Analytics & Dashboarding

Create Power BI dashboards (via Gold views only) covering:

- Omni-Channel Sales Overview
- Store vs Online Performance

All Information within this document is Intellectual property of NIIT (NIIT Ltd). No part of this document or the program design or program structure mentioned within can be shared or used within any organization without the permission of NIIT (NIIT Ltd).

- Category & Brand Performance
- Customer Segmentation & Loyalty Analysis
- Inventory Stock-out Risk

9. Data Validation & Quality Checks

Implement executable business rules with reject table and rule logging:

- quantity > 0
- total_amount >= 0
- selling_price >= cost_price
- stock_on_hand >= 0

10. Monitoring & Alerting Setup

- Configure Airflow on_failure_callback with email or Slack notification simulation
- Set SLA thresholds on each DAG task and configure sla_miss_callback
- Implement retry exhaustion escalation alert
- Connect metadata logging output to a pipeline monitoring dashboard view

Key Analytical Use Cases

1. Omni-Channel Sales Performance

- Compare revenue across Store, Online, and Mobile
- Identify channel contribution %

2. Product & Category Insights

- Top-selling products by revenue and quantity
- Category-wise margin analysis

3. Store Performance Analysis

- Store-level revenue, footfall proxy (orders)
- City and region performance comparison

4. Customer Analytics

- Repeat vs first-time customers
- Loyalty tier contribution to revenue
- Average Order Value by segment

5. Inventory Optimization

All Information within this document is Intellectual property of NIIT (NIIT Ltd). No part of this document or the program design or program structure mentioned within can be shared or used within any organization without the permission of NIIT (NIIT Ltd).

- Identify products below reorder level
- Slow-moving vs fast-moving SKUs

All Information within this document is Intellectual property of NIIT (NIIT Ltd). No part of this document or the program design or program structure mentioned within can be shared or used within any organization without the permission of NIIT (NIIT Ltd).